DSA TREES — PROBLEM SET 2

60 complete coding problems • 20 Easy • 20 Medium • 20 Hard • Java solutions

Format: Problem → Example → Approach → Algorithm → Java Code → Complexity

Easy Problems — 20

1. Count Nodes in a Binary Tree

Problem: Return the total number of nodes in a binary tree.

Example Input: root=[1,2,3,4,5]

Example Output: 5

Approach: Visit every node once and add one.

Algorithm: Return 0 for null; otherwise return 1 + left count + right count.

Java Answer:

```
class Solution {
    public int countNodes(TreeNode root) {
        if (root == null) return 0;
        return 1 + countNodes(root.left) + countNodes(root.right);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

2. Sum of All Binary Tree Nodes

Problem: Return the sum of every value in the tree.

Example Input: root=[1,2,3]

Example Output: 6

Approach: Use recursive subtree sums.

Algorithm: Return node value plus sums of both children.

Java Answer:

```
class Solution {
    public int sumNodes(TreeNode root) {
        if (root == null) return 0;
        return root.val + sumNodes(root.left) + sumNodes(root.right);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

3. Count Leaf Nodes

Problem: Count nodes that have no children.

Example Input: root=[1,2,3,4,5]

Example Output: 3

Approach: A leaf has both child pointers null.

Algorithm: Return 1 for a leaf; otherwise count leaves in both subtrees.

Java Answer:

```
class Solution {
    public int countLeaves(TreeNode root) {
        if (root == null) return 0;
        if (root.left == null && root.right == null) return 1;
        return countLeaves(root.left) + countLeaves(root.right);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

4. Count Full Nodes

Problem: Count nodes having exactly two children.

Example Input: root=[1,2,3,4,5]

Example Output: 2

Approach: Check whether both child pointers are non-null.

Algorithm: Add one when both children exist and recurse.

Java Answer:

```
class Solution {
    public int countFull(TreeNode root) {
        if (root == null) return 0;
        int here = (root.left != null && root.right != null) ? 1 : 0;
        return here + countFull(root.left) + countFull(root.right);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

5. Find Maximum Value in Binary Tree

Problem: Find the largest value without assuming BST ordering.

Example Input: root=[5,1,9,3]

Example Output: 9

Approach: Compare current value with maximums from both subtrees.

Algorithm: Return negative infinity for null and combine three candidates.

Java Answer:

```
class Solution {
    public int maxValue(TreeNode root) {
        if (root == null) return Integer.MIN_VALUE;
        return Math.max(root.val,
            Math.max(maxValue(root.left), maxValue(root.right)));
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

6. Find Minimum Value in Binary Tree

Problem: Find the smallest value in a general binary tree.

Example Input: root=[5,1,9,3]

Example Output: 1

Approach: Unlike a BST, both subtrees must be searched.

Algorithm: Take minimum of current value and subtree minima.

Java Answer:

```
class Solution {
    public int minValue(TreeNode root) {
        if (root == null) return Integer.MAX_VALUE;
        return Math.min(root.val,
            Math.min(minValue(root.left), minValue(root.right)));
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

7. Nodes at a Given Level

Problem: Print or return all nodes at level k, where root is level 0.

Example Input: root=[1,2,3,4,5], k=2

Example Output: [4,5]

Approach: Decrease the level as recursion moves downward.

Algorithm: When k becomes zero, record the current node.

Java Answer:

```
class Solution {
    public List<Integer> nodesAtLevel(TreeNode root, int k) {
        List<Integer> ans = new ArrayList<>();
        dfs(root, k, ans);
        return ans;
    }
    private void dfs(TreeNode n, int k, List<Integer> ans) {
        if (n == null) return;
        if (k == 0) { ans.add(n.val); return; }
        dfs(n.left, k - 1, ans);
        dfs(n.right, k - 1, ans);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

8. Find the Deepest Node

Problem: Return the last node at the maximum depth.

Example Input: root=[1,2,3,4,5]

Example Output: 5

Approach: BFS leaves the deepest level in the queue last.

Algorithm: Process the queue until empty and keep the last removed node.

Java Answer:

```
class Solution {
    public int deepestNode(TreeNode root) {
        if (root == null) throw new IllegalArgumentException();
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);
        TreeNode last = root;
        while (!q.isEmpty()) {
            last = q.poll();
            if (last.left != null) q.offer(last.left);
            if (last.right != null) q.offer(last.right);
        }
        return last.val;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

9. Number of Levels in a Tree

Problem: Return how many levels the tree contains.

Example Input: root=[1,2,3,4]

Example Output: 3

Approach: The number of levels equals maximum depth.

Algorithm: Use BFS and increment a counter once per processed level.

Java Answer:

```
class Solution {
    public int levels(TreeNode root) {
        if (root == null) return 0;
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);
        int levels = 0;
        while (!q.isEmpty()) {
            int size = q.size();
            while (size-- > 0) {
                TreeNode n = q.poll();
                if (n.left != null) q.offer(n.left);
                if (n.right != null) q.offer(n.right);
            }
            levels++;
        }
        return levels;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

10. Count Nodes Greater Than X

Problem: Count nodes whose values are strictly greater than x.

Example Input: root=[5,2,8,1,3], x=4

Example Output: 2

Approach: Visit every node and test its value.

Algorithm: Add one when node.val > x and recurse.

Java Answer:

```
class Solution {
    public int countGreater(TreeNode root, int x) {
        if (root == null) return 0;
        return (root.val > x ? 1 : 0)
             + countGreater(root.left, x)
             + countGreater(root.right, x);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

11. Check Children Sum Property

Problem: Every non-leaf node must equal the sum of its existing children.

Example Input: root=[10,8,2,3,5]

Example Output: true

Approach: Leaves are valid automatically; check each internal node.

Algorithm: Compare node value with left value plus right value, treating null as zero.

Java Answer:

```
class Solution {
    public boolean checkChildrenSum(TreeNode root) {
        if (root == null || (root.left == null && root.right == null))
            return true;
        int sum = 0;
        if (root.left != null) sum += root.left.val;
        if (root.right != null) sum += root.right.val;
        return root.val == sum
            && checkChildrenSum(root.left)
            && checkChildrenSum(root.right);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

12. Check Identical Trees Iteratively

Problem: Determine whether two trees are identical using BFS.

Example Input: p=[1,2,3], q=[1,2,3]

Example Output: true

Approach: Use a queue containing corresponding node pairs.

Algorithm: For every pair compare null status and values, then enqueue children.

Java Answer:

```
class Solution {
    public boolean sameTree(TreeNode a, TreeNode b) {
        Queue<TreeNode> q1 = new LinkedList<>();
        Queue<TreeNode> q2 = new LinkedList<>();
        q1.offer(a); q2.offer(b);

        while (!q1.isEmpty()) {
            TreeNode x = q1.poll(), y = q2.poll();
            if (x == null || y == null) {
                if (x != y) return false;
                continue;
            }
            if (x.val != y.val) return false;
            q1.offer(x.left); q1.offer(x.right);
            q2.offer(y.left); q2.offer(y.right);
        }
        return true;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

13. Mirror a Binary Tree

Problem: Create the mirror image of a binary tree.

Example Input: root=[1,2,3,4,5]

Example Output: [1,3,2,null,null,5,4]

Approach: Swap children recursively.

Algorithm: Swap left and right before recursively processing both.

Java Answer:

```
class Solution {
    public TreeNode mirror(TreeNode root) {
        if (root == null) return null;
        TreeNode t = root.left;
        root.left = root.right;
        root.right = t;
        mirror(root.left);
        mirror(root.right);
        return root;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

14. Find Parent of a Target Node

Problem: Return the parent value of a target node in a binary tree.

Example Input: root=[1,2,3,4,5], target=5

Example Output: 2

Approach: Carry the parent during DFS.

Algorithm: If either child matches target, return current; otherwise search subtrees.

Java Answer:

```
class Solution {
    public TreeNode findParent(TreeNode root, int target) {
        return dfs(root, target);
    }
    private TreeNode dfs(TreeNode n, int x) {
        if (n == null) return null;
        if ((n.left != null && n.left.val == x) ||
            (n.right != null && n.right.val == x)) return n;
        TreeNode a = dfs(n.left, x);
        return a != null ? a : dfs(n.right, x);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

15. Average of Each Level

Problem: Return the average value of nodes at every level.

Example Input: root=[3,9,20,null,null,15,7]

Example Output: [3.0,14.5,11.0]

Approach: BFS isolates each level, allowing a simple sum divided by its size.

Algorithm: For each level, sum all node values and divide by number of nodes.

Java Answer:

```
class Solution {
    public List<Double> averageOfLevels(TreeNode root) {
        List<Double> ans = new ArrayList<>();
        if (root == null) return ans;
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);

        while (!q.isEmpty()) {
            int size = q.size();
            long sum = 0;
            for (int i = 0; i < size; i++) {
                TreeNode n = q.poll();
                sum += n.val;
                if (n.left != null) q.offer(n.left);
                if (n.right != null) q.offer(n.right);
            }
            ans.add((double) sum / size);
        }
        return ans;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

16. Find Second Largest Value

Problem: Find the second-largest distinct value in a binary tree.

Example Input: root=[2,2,5,5,7]

Example Output: 5

Approach: Track the largest and second-largest distinct values while traversing.

Algorithm: Update max and second when a larger distinct value is found; update second for intermediate values.

Java Answer:

```
class Solution {
    public int secondLargest(TreeNode root) {
        long[] best = {Long.MIN_VALUE, Long.MIN_VALUE};
        dfs(root, best);
        if (best[1] == Long.MIN_VALUE) throw new IllegalArgumentException();
        return (int) best[1];
    }
    private void dfs(TreeNode n, long[] b) {
        if (n == null) return;
        if (n.val > b[0]) { b[1] = b[0]; b[0] = n.val; }
        else if (n.val < b[0] && n.val > b[1]) b[1] = n.val;
        dfs(n.left, b); dfs(n.right, b);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

17. Find Second Smallest Value

Problem: Find the second-smallest distinct value in a binary tree.

Example Input: root=[2,2,5,1,7]

Example Output: 2

Approach: Track the two smallest distinct values.

Algorithm: Update minimum and second minimum during DFS.

Java Answer:

```
class Solution {
    public int secondSmallest(TreeNode root) {
        long[] b = {Long.MAX_VALUE, Long.MAX_VALUE};
        dfs(root, b);
        if (b[1] == Long.MAX_VALUE) throw new IllegalArgumentException();
        return (int) b[1];
    }
    private void dfs(TreeNode n, long[] b) {
        if (n == null) return;
        if (n.val < b[0]) { b[1] = b[0]; b[0] = n.val; }
        else if (n.val > b[0] && n.val < b[1]) b[1] = n.val;
        dfs(n.left, b); dfs(n.right, b);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

18. Find Level with Maximum Sum

Problem: Return the level having the maximum sum of values.

Example Input: root=[1,7,0,7,-8,null,null]

Example Output: 2

Approach: Use BFS and compare each level sum.

Algorithm: Maintain level number and best sum while processing each level.

Java Answer:

```
class Solution {
    public int maxLevelSum(TreeNode root) {
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);
        int level = 0, bestLevel = 1;
        long best = Long.MIN_VALUE;

        while (!q.isEmpty()) {
            level++;
            int size = q.size();
            long sum = 0;
            while (size-- > 0) {
                TreeNode n = q.poll();
                sum += n.val;
                if (n.left != null) q.offer(n.left);
                if (n.right != null) q.offer(n.right);
            }
            if (sum > best) { best = sum; bestLevel = level; }
        }
        return bestLevel;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

Medium Problems — 20

1. Boundary Traversal of Binary Tree

Problem: Return root, left boundary, leaves, and reversed right boundary without duplicates.

Example Input: root=[1,2,3,4,5,6,7]

Example Output: [1,2,4,5,6,7,3]

Approach: Split the boundary into four controlled parts.

Algorithm: Add root; traverse left boundary excluding leaves; add all leaves; traverse right boundary excluding leaves in reverse.

Java Answer:

```
class Solution {
    public List<Integer> boundary(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        if (root == null) return ans;
        if (!isLeaf(root)) ans.add(root.val);
        addLeft(root.left, ans);
        addLeaves(root, ans);
        List<Integer> right = new ArrayList<>();
        addRight(root.right, right);
        Collections.reverse(right);
        ans.addAll(right);
        return ans;
    }
    private boolean isLeaf(TreeNode n) {
        return n != null && n.left == null && n.right == null;
    }
    private void addLeft(TreeNode n, List<Integer> a) {
        while (n != null) {
            if (!isLeaf(n)) a.add(n.val);
            n = n.left != null ? n.left : n.right;
        }
    }
    private void addLeaves(TreeNode n, List<Integer> a) {
        if (n == null) return;
        if (isLeaf(n)) { a.add(n.val); return; }
        addLeaves(n.left, a); addLeaves(n.right, a);
    }
    private void addRight(TreeNode n, List<Integer> a) {
        while (n != null) {
            if (!isLeaf(n)) a.add(n.val);
            n = n.right != null ? n.right : n.left;
        }
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h) excluding output

2. Top View of Binary Tree

Problem: Return the first node visible at each horizontal distance from the root.

Example Input: root=[1,2,3,null,4,null,5]

Example Output: [2,1,3,5]

Approach: BFS visits nodes top-down, so the first node at each horizontal distance is visible.

Algorithm: Track horizontal distance with each queue item and store it only if unseen.

Java Answer:

```
class Solution {
    static class Pair { TreeNode n; int hd; Pair(TreeNode n,int h){this.n=n;hd=h;} }

    public List<Integer> topView(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        if (root == null) return ans;
        Map<Integer,Integer> map = new TreeMap<>();
        Queue<Pair> q = new LinkedList<>();
        q.offer(new Pair(root, 0));

        while (!q.isEmpty()) {
            Pair p = q.poll();
            if (!map.containsKey(p.hd)) map.put(p.hd, p.n.val);
            if (p.n.left != null) q.offer(new Pair(p.n.left, p.hd - 1));
            if (p.n.right != null) q.offer(new Pair(p.n.right, p.hd + 1));
        }
        ans.addAll(map.values());
        return ans;
    }
}
```

Time Complexity: O(n log n) **Space Complexity:** O(n)

3. Bottom View of Binary Tree

Problem: Return the visible node at each horizontal distance when viewed from below.

Example Input: root=[1,2,3,4,5,6,7]

Example Output: [4,2,6,3,7]

Approach: BFS visits nodes top-down; replacing the value at each horizontal distance leaves the lowest encountered node.

Algorithm: Track horizontal distance and overwrite the map value for every visited node.

Java Answer:

```
class Solution {
    static class Pair { TreeNode n; int hd; Pair(TreeNode n,int h){this.n=n;hd=h;} }

    public List<Integer> bottomView(TreeNode root) {
        Map<Integer,Integer> map = new TreeMap<>();
        Queue<Pair> q = new LinkedList<>();
        q.offer(new Pair(root,0));
        while (!q.isEmpty()) {
            Pair p=q.poll();
            map.put(p.hd,p.n.val);
            if(p.n.left!=null) q.offer(new Pair(p.n.left,p.hd-1));
            if(p.n.right!=null) q.offer(new Pair(p.n.right,p.hd+1));
        }
        return new ArrayList<>(map.values());
    }
}
```

Time Complexity: O(n log n) **Space Complexity:** O(n)

4. Vertical Order Traversal

Problem: Group nodes by column from left to right, preserving top-to-bottom ordering and required tie breaking by value.

Example Input: root=[3,9,20,null,null,15,7]

Example Output: [[9],[3,15],[20],[7]]

Approach: Store nodes by column and sort by row/value when multiple nodes share positions.

Algorithm: DFS records (column,row,value), then sort all records and group by column.

Java Answer:

```
class Solution {
    static class Item {
        int col,row,val;
        Item(int c,int r,int v){col=c;row=r;val=v;}
    }

    public List<List<Integer>> verticalTraversal(TreeNode root) {
        List<Item> all = new ArrayList<>();
        dfs(root,0,0,all);
        all.sort((a,b) -> {
            if (a.col != b.col) return Integer.compare(a.col,b.col);
            if (a.row != b.row) return Integer.compare(a.row,b.row);
            return Integer.compare(a.val,b.val);
        });

        List<List<Integer>> ans = new ArrayList<>();
        int prev = Integer.MIN_VALUE;
        for (Item x : all) {
            if (ans.isEmpty() || x.col != prev) ans.add(new ArrayList<>());
            ans.get(ans.size()-1).add(x.val);
            prev = x.col;
        }
        return ans;
    }

    private void dfs(TreeNode n,int c,int r,List<Item> a){
        if(n==null)return;
        a.add(new Item(c,r,n.val));
        dfs(n.left,c-1,r+1,a);
        dfs(n.right,c+1,r+1,a);
    }
}
```

Time Complexity: O(n log n) **Space Complexity:** O(n)

5. Diagonal Traversal

Problem: Group nodes by diagonal distance, where moving right stays on the diagonal and moving left increases it.

Example Input: root=[8,3,10,1,6,null,14]

Example Output: [[8,10,14],[3,6],[1]]

Approach: Assign each node a diagonal index. BFS/DFS can carry that index.

Algorithm: Right child keeps diagonal; left child increments diagonal.

Java Answer:

```
class Solution {
    public List<List<Integer>> diagonal(TreeNode root) {
        Map<Integer,List<Integer>> map = new TreeMap<>();
        dfs(root,0,map);
        return new ArrayList<>(map.values());
    }
    private void dfs(TreeNode n,int d,Map<Integer,List<Integer>> map){
        if(n==null)return;
        map.computeIfAbsent(d,k->new ArrayList<>()).add(n.val);
        dfs(n.left,d+1,map);
        dfs(n.right,d,map);
    }
}
```

Time Complexity: O(n log n) **Space Complexity:** O(n)

6. BST Inorder Successor

Problem: Given a BST and a node, find the smallest value strictly greater than the node.

Example Input: root=[5,3,6,2,4], p=4

Example Output: 5

Approach: While traversing, a node larger than p is a successor candidate; move left to find a smaller candidate.

Algorithm: If p.val < current, save current and go left; otherwise go right.

Java Answer:

```
class Solution {
    public TreeNode successor(TreeNode root, TreeNode p) {
        TreeNode ans = null;
        while (root != null) {
            if (p.val < root.val) {
                ans = root;
                root = root.left;
            } else {
                root = root.right;
            }
        }
        return ans;
    }
}
```

Time Complexity: O(h) **Space Complexity:** O(1)

7. BST Inorder Predecessor

Problem: Given a BST and a node, find the largest value strictly smaller than the node.

Example Input: root=[5,3,6,2,4], p=4

Example Output: 3

Approach: A smaller current value is a predecessor candidate; move right to improve it.

Algorithm: If p.val > current, save current and go right; otherwise go left.

Java Answer:

```
class Solution {
    public TreeNode predecessor(TreeNode root, TreeNode p) {
        TreeNode ans = null;
        while (root != null) {
            if (p.val > root.val) {
                ans = root;
                root = root.right;
            } else {
                root = root.left;
            }
        }
        return ans;
    }
}
```

Time Complexity: O(h) **Space Complexity:** O(1)

8. Trim a BST

Problem: Remove all nodes whose values are outside [low,high], preserving BST structure.

Example Input: root=[3,0,4,null,2,null,null,1], low=1, high=3

Example Output: [3,2,null,1]

Approach: BST ordering tells which entire branch can be discarded.

Algorithm: If node is below low return trimmed right; if above high return trimmed left; otherwise trim both children.

Java Answer:

```
class Solution {
    public TreeNode trimBST(TreeNode root, int low, int high) {
        if (root == null) return null;
        if (root.val < low) return trimBST(root.right, low, high);
        if (root.val > high) return trimBST(root.left, low, high);
        root.left = trimBST(root.left, low, high);
        root.right = trimBST(root.right, low, high);
        return root;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

9. Convert BST to Greater Sum Tree

Problem: Replace each node by itself plus the sum of all greater values.

Example Input: root=[4,1,6,0,2,5,7]

Example Output: greater-sum transformed BST

Approach: Reverse inorder visits nodes from largest to smallest.

Algorithm: Maintain a running sum and replace each node value with running sum.

Java Answer:

```
class Solution {
    private int sum = 0;

    public TreeNode bstToGst(TreeNode root) {
        dfs(root);
        return root;
    }
    private void dfs(TreeNode n) {
        if(n==null)return;
        dfs(n.right);
        sum += n.val;
        n.val = sum;
        dfs(n.left);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

10. Two Sum in a BST

Problem: Determine whether two distinct nodes in a BST sum to target.

Example Input: root=[5,3,6,2,4,null,7], target=9

Example Output: true

Approach: Use a set while traversing, or exploit sorted inorder. A hash set gives simple O(n).

Algorithm: For each value x, check whether target-x has already been seen.

Java Answer:

```
class Solution {
    public boolean findTarget(TreeNode root, int k) {
        Set<Integer> seen = new HashSet<>();
        return dfs(root,k,seen);
    }
    private boolean dfs(TreeNode n,int k,Set<Integer> s){
        if(n==null)return false;
        if(s.contains(k-n.val))return true;
        s.add(n.val);
        return dfs(n.left,k,s) || dfs(n.right,k,s);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

11. Count Paths with Given Sum

Problem: Count downward paths whose values add to target; a path can start at any node.

Example Input: root=[10,5,-3,3,2,null,11,3,-2,null,1], target=8

Example Output: 3

Approach: Prefix sums let us count how many earlier path sums produce the required difference.

Algorithm: At each node, currentPrefix-target tells how many valid starting points exist.

Java Answer:

```
class Solution {
    public int pathSum(TreeNode root, int targetSum) {
        Map<Long,Integer> freq = new HashMap<>();
        freq.put(0L,1);
        return dfs(root,0,targetSum,freq);
    }
    private int dfs(TreeNode n,long sum,int target,Map<Long,Integer> f){
        if(n==null)return 0;
        sum += n.val;
        int ans = f.getOrDefault(sum-target,0);
        f.put(sum,f.getOrDefault(sum,0)+1);
        ans += dfs(n.left,sum,target,f);
        ans += dfs(n.right,sum,target,f);
        f.put(sum,f.get(sum)-1);
        return ans;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

12. Find Modes in a BST

Problem: Return all values that occur most frequently in a BST.

Example Input: root=[1,null,2,2]

Example Output: [2]

Approach: Inorder traversal groups equal values consecutively, so frequencies can be counted with O(h) extra space.

Algorithm: Track current run and maximum frequency; update the answer list whenever the frequency changes.

Java Answer:

```
class Solution {
    private Integer prev = null;
    private int count = 0, max = 0;
    private List<Integer> ans = new ArrayList<>();

    public int[] findMode(TreeNode root) {
        inorder(root);
        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
    private void inorder(TreeNode n) {
        if(n==null)return;
        inorder(n.left);
        if(prev != null && prev == n.val) count++;
        else count = 1;
        prev = n.val;
        if(count > max) { max=count; ans.clear(); ans.add(n.val); }
        else if(count == max) ans.add(n.val);
        inorder(n.right);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h) excluding output

13. Build Balanced BST from Sorted Linked List

Problem: Convert a sorted singly linked list to a height-balanced BST.

Example Input: head=[-10,-3,0,5,9]

Example Output: balanced BST

Approach: Use slow/fast pointers to find the middle node as root.

Algorithm: Find middle, detach it, recursively build left and right halves.

Java Answer:

```
class Solution {
    public TreeNode sortedListToBST(ListNode head) {
        if (head == null) return null;
        if (head.next == null) return new TreeNode(head.val);

        ListNode prev = null, slow = head, fast = head;
        while (fast != null && fast.next != null) {
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;
        }

        prev.next = null;
        TreeNode root = new TreeNode(slow.val);
        root.left = sortedListToBST(head);
        root.right = sortedListToBST(slow.next);
        return root;
    }
}
```

Time Complexity: O(n log n) **Space Complexity:** O(log n)

14. Minimum Height BST from Sorted Array

Problem: Construct a BST with minimum possible height from a sorted array.

Example Input: nums=[1,2,3,4,5,6,7]

Example Output: height 3

Approach: Choose the middle element at every recursive step.

Algorithm: Midpoint becomes root; recurse on both halves.

Java Answer:

```
class Solution {
    public TreeNode build(int[] a) {
        return build(a,0,a.length-1);
    }
    private TreeNode build(int[] a,int l,int r){
        if(l>r)return null;
        int m=l+(r-l)/2;
        TreeNode n=new TreeNode(a[m]);
        n.left=build(a,l,m-1);
        n.right=build(a,m+1,r);
        return n;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(log n)

15. BST Range Search

Problem: Return all BST values within [low,high] in sorted order.

Example Input: root=[10,5,15,3,7,13,18], low=6, high=16

Example Output: [7,10,13,15]

Approach: Inorder gives sorted output, while BST bounds prune irrelevant branches.

Algorithm: If current is above low, search left; include current when in range; if below high, search right.

Java Answer:

```
class Solution {
    public List<Integer> range(TreeNode root,int low,int high){
        List<Integer> ans=new ArrayList<>();
        dfs(root,low,high,ans);
        return ans;
    }
    private void dfs(TreeNode n,int l,int h,List<Integer> a){
        if(n==null)return;
        if(n.val>l) dfs(n.left,l,h,a);
        if(n.val>=l && n.val<=h) a.add(n.val);
        if(n.val<h) dfs(n.right,l,h,a);
    }
}
```

Time Complexity: O(h+k) typical, O(n) worst **Space Complexity:** O(h)

Hard Problems — 20

1. Tree Diameter with Two DFS

Problem: Given an undirected tree represented by edges, find its diameter length in edges.

Example Input: n=6, edges=[[0,1],[1,2],[1,3],[3,4],[4,5]]

Example Output: 4

Approach: Starting from any node, a farthest node is an endpoint of a diameter. A second traversal from that endpoint finds the diameter.

Algorithm: DFS/BFS from node 0 to find farthest A; traverse from A to find farthest B and its distance.

Java Answer:

```
class Solution {
    public int diameter(int n, int[][] edges) {
        List<Integer>[] g = new ArrayList[n];
        for(int i=0;i<n;i++) g[i]=new ArrayList<>();
        for(int[] e:edges){g[e[0]].add(e[1]);g[e[1]].add(e[0]);}

        int[] a = farthest(0,g);
        int[] b = farthest(a[0],g);
        return b[1];
    }

    private int[] farthest(int start,List<Integer>[] g){
        int[] dist=new int[g.length];
        Arrays.fill(dist,-1);
        Queue<Integer> q=new LinkedList<>();
        q.offer(start); dist[start]=0;
        int node=start;
        while(!q.isEmpty()){
            int u=q.poll();
            if(dist[u]>dist[node]) node=u;
            for(int v:g[u]) if(dist[v]==-1){
                dist[v]=dist[u]+1;
                q.offer(v);
            }
        }
        return new int[]{node,dist[node]};
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

2. Minimum Time to Collect All Apples

Problem: In a tree rooted at 0, some nodes contain apples. Find the minimum number of edge traversals needed to collect all apples and return to root.

Example Input: n=7, edges=[[0,1],[0,2],[1,4],[1,5],[2,3],[2,6]], hasApple=[false,false,true,false,true,true,false]

Example Output: 6

Approach: Any subtree containing an apple requires traversing its connecting edge twice: down and back.

Algorithm: DFS children. If a child subtree contains an apple, add childCost+2.

Java Answer:

```
class Solution {
    public int minTime(int n,int[][] edges,List<Boolean> apple){
        List<Integer>[] g=new ArrayList[n];
        for(int i=0;i<n;i++)g[i]=new ArrayList<>();
        for(int[] e:edges){g[e[0]].add(e[1]);g[e[1]].add(e[0]);}
        return dfs(0,-1,g,apple);
    }
    private int dfs(int u,int p,List<Integer>[] g,List<Boolean> a){
        int cost=0;
        for(int v:g[u]) if(v!=p){
            int sub=dfs(v,u,g,a);
            if(sub>0 || a.get(v)) cost += sub+2;
        }
        return cost;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

3. Longest Univalue Path

Problem: Find the longest path where every node has the same value.

Example Input: root=[5,4,5,1,1,null,5]

Example Output: 2

Approach: For each node, calculate matching downward chains from left and right. A path through the node can combine both.

Algorithm: If child value matches, its chain length contributes; update global answer with leftChain+rightChain.

Java Answer:

```
class Solution {
    private int ans=0;
    public int longestUnivaluePath(TreeNode root){
        dfs(root);
        return ans;
    }
    private int dfs(TreeNode n){
        if(n==null)return 0;
        int l=dfs(n.left), r=dfs(n.right);
        int left=0,right=0;
        if(n.left!=null && n.left.val==n.val) left=l+1;
        if(n.right!=null && n.right.val==n.val) right=r+1;
        ans=Math.max(ans,left+right);
        return Math.max(left,right);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

4. Maximum Sum Path from Leaf to Leaf

Problem: Find the maximum sum of a path connecting two leaves in a binary tree. Return the maximum downward leaf path when no two-leaf path exists.

Example Input: root=[-15,5,6,-8,1,3,9,2,6,-4,0]

Example Output: 27

Approach: At each node combine best leaf-to-node sums from both children if both exist.

Algorithm: Postorder returns best downward leaf sum and updates global answer when two child branches exist.

Java Answer:

```
class Solution {
    private long ans=Long.MIN_VALUE;
    public long maxLeafToLeaf(TreeNode root){
        dfs(root);
        return ans;
    }
    private long dfs(TreeNode n){
        if(n==null)return Long.MIN_VALUE/4;
        if(n.left==null && n.right==null)return n.val;
        long l=dfs(n.left), r=dfs(n.right);
        if(n.left!=null && n.right!=null)
            ans=Math.max(ans,l+r+n.val);
        return n.val + Math.max(l,r);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

5. Maximum Matching in a Tree

Problem: Find the largest set of edges in a tree such that no two selected edges share an endpoint.

Example Input: edges=[[0,1],[1,2],[1,3],[3,4]]

Example Output: 2

Approach: Root the tree and use DP: either match the current node with one child or do not match it.

Algorithm: dp0 = best when node is not matched to a child; dp1 = best when it is. Evaluate each possible child match.

Java Answer:

```
class Solution {
    List<Integer>[] g;
    public int maxMatching(int n,int[][] edges){
        g=new ArrayList[n];
        for(int i=0;i<n;i++)g[i]=new ArrayList<>();
        for(int[] e:edges){g[e[0]].add(e[1]);g[e[1]].add(e[0]);}
        int[] d=dfs(0,-1);
        return Math.max(d[0],d[1]);
    }
    // [not matched to child, matched to one child]
    private int[] dfs(int u,int p){
        int base=0;
        List<int[]> child=new ArrayList<>();
        for(int v:g[u]) if(v!=p){
            int[] d=dfs(v,u);
            base += Math.max(d[0],d[1]);
            child.add(d);
        }
        int best=base;
        for(int[] d:child)
            best=Math.max(best,1+d[0]+base-Math.max(d[0],d[1]));
        return new int[]{base,best};
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

6. Minimum Vertex Cover on an Undirected Tree

Problem: Find the minimum number of vertices needed so every edge has at least one selected endpoint.

Example Input: edges=[[0,1],[0,2],[1,3],[1,4]]

Example Output: 2

Approach: For each node, compute minimum cover when it is selected or not selected.

Algorithm: If node is skipped, all children must be selected. If selected, each child chooses its cheaper state.

Java Answer:

```
class Solution {
    List<Integer>[] g;
    public int minCover(int n,int[][] edges){
        g=new ArrayList[n];
        for(int i=0;i<n;i++)g[i]=new ArrayList<>();
        for(int[] e:edges){g[e[0]].add(e[1]);g[e[1]].add(e[0]);}
        int[] d=dfs(0,-1);
        return Math.min(d[0],d[1]);
    }
    // [take, skip]
    private int[] dfs(int u,int p){
        int take=1,skip=0;
        for(int v:g[u]) if(v!=p){
            int[] d=dfs(v,u);
            take += Math.min(d[0],d[1]);
            skip += d[0];
        }
        return new int[]{take,skip};
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

7. Reroot Tree Height for Every Node

Problem: Given an undirected tree, compute the height when every node is considered the root.

Example Input: edges=[[0,1],[0,2],[1,3],[1,4]]

Example Output: [2,2,3,3,3]

Approach: Use two passes. First calculate downward heights; second propagate the best height coming from the parent side.

Algorithm: For each child, its outside height is max(parentOutside, siblingDownward)+1.

Java Answer:

```
class Solution {
    List<Integer>[] g;
    int[] down, up, ans;

    public int[] allHeights(int n,int[][] edges){
        g=new ArrayList[n];
        for(int i=0;i<n;i++)g[i]=new ArrayList<>();
        for(int[] e:edges){g[e[0]].add(e[1]);g[e[1]].add(e[0]);}
        down=new int[n]; up=new int[n]; ans=new int[n];
        dfs1(0,-1); dfs2(0,-1);
        for(int i=0;i<n;i++) ans[i]=Math.max(down[i],up[i]);
        return ans;
    }
    private void dfs1(int u,int p){
        for(int v:g[u])if(v!=p){
            dfs1(v,u);
            down[u]=Math.max(down[u],down[v]+1);
        }
    }
    private void dfs2(int u,int p){
        int best1=-1,best2=-1,bestChild=-1;
        for(int v:g[u])if(v!=p){
            int x=down[v]+1;
            if(x>best1){best2=best1;best1=x;bestChild=v;}
            else if(x>best2)best2=x;
        }
        for(int v:g[u])if(v!=p){
            int use=(bestChild==v?best2:best1);
            up[v]=1+Math.max(up[u],Math.max(0,use));
            dfs2(v,u);
        }
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

8. Lowest Common Ancestor with Binary Lifting

Problem: Preprocess a tree so LCA queries can be answered in O(log n).

Example Input: n=7, edges form a rooted tree, query=(4,6)

Example Output: 0

Approach: Binary lifting stores 2^j-th ancestors and uses depth equalization followed by simultaneous jumps.

Algorithm: Lift the deeper node, then lift both nodes from the highest power down until their parents match.

Java Answer:

```
class LCA {
    int LOG;
    int[][] up;
    int[] depth;
    List<Integer>[] g;

    LCA(int n,int[][] edges){
        LOG=1;
        while((1<<LOG)<=n)LOG++;
        up=new int[n][LOG];
        depth=new int[n];
        g=new ArrayList[n];
        for(int i=0;i<n;i++)g[i]=new ArrayList<>();
        for(int[] e:edges){g[e[0]].add(e[1]);g[e[1]].add(e[0]);}
        Arrays.fill(up[0],-1);
        dfs(0,-1);
    }
    void dfs(int u,int p){
        up[u][0]=p;
        for(int j=1;j<LOG;j++){
            int a=up[u][j-1];
            up[u][j]=(a==-1?-1:up[a][j-1]);
        }
        for(int v:g[u])if(v!=p){
            depth[v]=depth[u]+1;
            dfs(v,u);
        }
    }
    int query(int a,int b){
        if(depth[a]<depth[b]){int t=a;a=b;b=t;}
        int d=depth[a]-depth[b];
        for(int j=0;j<LOG;j++)if((d&(1<<j))!=0)a=up[a][j];
        if(a==b)return a;
        for(int j=LOG-1;j>=0;j--)
            if(up[a][j]!=up[b][j]){
                a=up[a][j]; b=up[b][j];
            }
        return up[a][0];
    }
}
```

Time Complexity: Preprocess O(n log n), query O(log n) **Space Complexity:** O(n log n)

9. Number of Unique BSTs

Problem: Given n distinct keys, count structurally unique BSTs that can be formed.

Example Input: n=3

Example Output: 5

Approach: If i is the root, i-1 keys go left and n-i go right. Multiply possibilities and sum.

Algorithm: Use Catalan DP: dp[n] = sum(dp[left]*dp[right]).

Java Answer:

```
class Solution {
    public int numTrees(int n) {
        long[] dp=new long[n+1];
        dp[0]=dp[1]=1;
        for(int nodes=2;nodes<=n;nodes++){
            for(int left=0;left<nodes;left++)
                dp[nodes]+=dp[left]*dp[nodes-1-left];
        }
        return (int)dp[n];
    }
}
```

Time Complexity: $O(n^2)$ **Space Complexity:** O(n)

10. Generate All Unique BSTs

Problem: Generate every structurally unique BST containing values 1..n.

Example Input: n=3

Example Output: 5 trees

Approach: Choose every possible root and recursively generate all left and right trees.

Algorithm: For each root r, combine every left tree from [l,r-1] with every right tree from [r+1,h].

Java Answer:

```
class Solution {
    public List<TreeNode> generateTrees(int n) {
        if(n==0)return new ArrayList<>();
        return build(1,n);
    }
    private List<TreeNode> build(int l,int r){
        List<TreeNode> res=new ArrayList<>();
        if(l>r){res.add(null);return res;}
        for(int root=l;root<=r;root++){
            for(TreeNode a:build(l,root-1))
                for(TreeNode b:build(root+1,r)){
                    TreeNode n=new TreeNode(root);
                    n.left=a;n.right=b;
                    res.add(n);
                }
        }
        return res;
    }
}
```

Time Complexity: O(Cn · n) output-sensitive **Space Complexity:** O(Cn · n) output-sensitive

11. Longest Path with Alternating Child Directions

Problem: Find the longest root-to-descendant path where the direction alternates left, right, left, right...

Example Input: root=[1,2,3,4,null,null,5]

Example Output: 2

Approach: Maintain two states representing the previous direction.

Algorithm: From a node reached by left, only right can continue the alternating path; similarly for right.

Java Answer:

```
class Solution {
    private int ans=0;
    public int longestAlternating(TreeNode root){
        dfs(root,0,0);
        return ans;
    }
    private void dfs(TreeNode n,int left,int right){
        if(n==null)return;
        ans=Math.max(ans,Math.max(left,right));
        dfs(n.left,right+1,0);
        dfs(n.right,0,left+1);
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

12. Tree Path Queries with Binary Lifting

Problem: Support kth-ancestor queries on a rooted tree after preprocessing.

Example Input: parent=[-1,0,0,1,1,2,2], query node=5,k=2

Example Output: 0

Approach: Binary lifting converts a large jump into O(log n) power-of-two jumps.

Algorithm: Build ancestor table and consume bits of k from low to high.

Java Answer:

```
class TreeQueries {
    int[][] up;
    int LOG;

    TreeQueries(int[] parent){
        int n=parent.length;
        LOG=1;
        while((1<<LOG)<=n)LOG++;
        up=new int[n][LOG];
        for(int i=0;i<n;i++)up[i][0]=parent[i];
        for(int j=1;j<LOG;j++)
            for(int i=0;i<n;i++){
                int p=up[i][j-1];
                up[i][j]=(p==-1?-1:up[p][j-1]);
            }
    }

    int kthAncestor(int node,int k){
        for(int j=0;j<LOG && node!=-1;j++)
            if((k&(1<<j))!=0) node=up[node][j];
        return node;
    }
}
```

Time Complexity: O(n log n) preprocessing, O(log n) query **Space Complexity:** O(n log n)

13. Maximum Independent Set on Weighted Tree

Problem: Each tree node has a value. Select nodes with maximum total value such that adjacent nodes cannot both be selected.

Example Input: values=[5,3,4,2,1]

Example Output: 9

Approach: Tree DP compares taking and skipping each node.

Algorithm: take=value+skip(children); skip=sum(max(take,skip) for children).

Java Answer:

```
class Solution {
    List<Integer>[] g;
    int[] val;

    public int maxIndependent(int[] values,int[][] edges){
        val=values;
        int n=values.length;
        g=new ArrayList[n];
        for(int i=0;i<n;i++)g[i]=new ArrayList<>();
        for(int[] e:edges){g[e[0]].add(e[1]);g[e[1]].add(e[0]);}
        int[] d=dfs(0,-1);
        return Math.max(d[0],d[1]);
    }
    private int[] dfs(int u,int p){
        int take=val[u],skip=0;
        for(int v:g[u])if(v!=p){
            int[] d=dfs(v,u);
            take+=d[1];
            skip+=Math.max(d[0],d[1]);
        }
        return new int[]{take,skip};
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

14. Tree Distance Sum for Every Node

Problem: For every node in a tree, calculate the sum of its distances to all other nodes.

Example Input: n=4, edges=[[0,1],[1,2],[1,3]]

Example Output: [5,3,5,5]

Approach: Compute the answer for root 0 using subtree sizes, then reroot each edge in O(1).

Algorithm: When moving from u to child v, nodes in v's subtree become one step closer and all others one step farther.

Java Answer:

```
class Solution {
    List<Integer>[] g;
    int[] size, ans;
    int n;

    public int[] distanceSums(int n,int[][] edges){
        this.n=n;
        g=new ArrayList[n];
        for(int i=0;i<n;i++)g[i]=new ArrayList<>();
        for(int[] e:edges){g[e[0]].add(e[1]);g[e[1]].add(e[0]);}
        size=new int[n]; ans=new int[n];
        dfs(0,-1);
        reroot(0,-1);
        return ans;
    }
    private void dfs(int u,int p){
        size[u]=1;
        for(int v:g[u])if(v!=p){
            dfs(v,u);
            size[u]+=size[v];
            ans[0]+=size[v];
        }
    }
    private void reroot(int u,int p){
        for(int v:g[u])if(v!=p){
            ans[v]=ans[u]-size[v]+n-size[v];
            reroot(v,u);
        }
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)

15. Minimum Cameras with Explicit Tree DP

Problem: Place the fewest cameras so every node is covered by itself, a child, or its parent.

Example Input: root=[0,0,null,0,0]

Example Output: 1

Approach: Use three-state postorder DP to distinguish camera, covered, and uncovered states.

Algorithm: A node needing coverage forces its parent to place a camera; otherwise a child camera covers the node.

Java Answer:

```
class Solution {
    private int cameras=0;
    public int minCameraCover(TreeNode root){
        if(state(root)==0)cameras++;
        return cameras;
    }
    private int state(TreeNode n){
        if(n==null)return 2;
        int l=state(n.left),r=state(n.right);
        if(l==0||r==0){cameras++;return 1;}
        if(l==1||r==1)return 2;
        return 0;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

16. Recover a BST Using O(1) Extra Space

Problem: Two values in a BST are swapped. Recover the tree using Morris inorder traversal instead of recursion or a stack.

Example Input: inorder=[1,3,2,4]

Example Output: inorder=[1,2,3,4]

Approach: Morris traversal temporarily creates predecessor links, reducing auxiliary space to O(1).

Algorithm: Detect inorder inversions while creating/removing threads; swap the first and second bad nodes.

Java Answer:

```
class Solution {
    public void recoverTree(TreeNode root) {
        TreeNode cur=root, prev=null, first=null, second=null;

        while(cur!=null){
            if(cur.left==null){
                if(prev!=null&&prev.val>cur.val){
                    if(first==null)first=prev;
                    second=cur;
                }
                prev=cur;
                cur=cur.right;
            }else{
                TreeNode p=cur.left;
                while(p.right!=null&&p.right!=cur)p=p.right;
                if(p.right==null){
                    p.right=cur;
                    cur=cur.left;
                }else{
                    p.right=null;
                    if(prev!=null&&prev.val>cur.val){
                        if(first==null)first=prev;
                        second=cur;
                    }
                    prev=cur;
                    cur=cur.right;
                }
            }
        }
        int t=first.val;first.val=second.val;second.val=t;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(1)

17. Morris Inorder Traversal

Problem: Return inorder traversal using O(1) auxiliary space.

Example Input: root=[1,null,2,3]

Example Output: [1,3,2]

Approach: Morris traversal uses the rightmost node of a left subtree as a temporary predecessor link.

Algorithm: Create a thread when first visiting a node with a left child; remove it on the second visit.

Java Answer:

```
class Solution {
    public List<Integer> inorder(TreeNode root){
        List<Integer> ans=new ArrayList<>();
        TreeNode cur=root;
        while(cur!=null){
            if(cur.left==null){
                ans.add(cur.val);
                cur=cur.right;
            }else{
                TreeNode p=cur.left;
                while(p.right!=null&&p.right!=cur)p=p.right;
                if(p.right==null){
                    p.right=cur;
                    cur=cur.left;
                }else{
                    p.right=null;
                    ans.add(cur.val);
                    cur=cur.right;
                }
            }
        }
        return ans;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(1)

18. Morris Preorder Traversal

Problem: Return preorder traversal using O(1) auxiliary space.

Example Input: root=[1,null,2,3]

Example Output: [1,2,3]

Approach: Morris traversal can visit the node when its predecessor thread is first created.

Algorithm: Record current before moving left on the first visit; on the second visit remove the thread.

Java Answer:

```
class Solution {
    public List<Integer> preorder(TreeNode root){
        List<Integer> ans=new ArrayList<>();
        TreeNode cur=root;
        while(cur!=null){
            if(cur.left==null){
                ans.add(cur.val);
                cur=cur.right;
            }else{
                TreeNode p=cur.left;
                while(p.right!=null&&p.right!=cur)p=p.right;
                if(p.right==null){
                    ans.add(cur.val);
                    p.right=cur;
                    cur=cur.left;
                }else{
                    p.right=null;
                    cur=cur.right;
                }
            }
        }
        return ans;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(1)

19. Minimum Depth with BFS

Problem: Find the minimum root-to-leaf depth using breadth-first search.

Example Input: root=[3,9,20,null,null,15,7]

Example Output: 2

Approach: BFS reaches the nearest leaf first, so the first leaf found gives the minimum depth.

Algorithm: Process levels and return immediately when a leaf is dequeued.

Java Answer:

```
class Solution {
    public int minDepth(TreeNode root){
        if(root==null)return 0;
        Queue<TreeNode> q=new LinkedList<>();
        q.offer(root);
        int depth=1;
        while(!q.isEmpty()){
            int size=q.size();
            while(size-->0){
                TreeNode n=q.poll();
                if(n.left==null&&n.right==null)return depth;
                if(n.left!=null)q.offer(n.left);
                if(n.right!=null)q.offer(n.right);
            }
            depth++;
        }
        return depth;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

20. Vertical Sum with Hash Map

Problem: Compute the sum of values in every vertical column.

Example Input: root=[1,2,3,4,5,6,7]

Example Output: [4,2,12,3,7]

Approach: Carry horizontal distance and accumulate node values by column.

Algorithm: DFS left decreases column and right increases it; use a sorted map for ordered columns.

Java Answer:

```
class Solution {
    public List<Integer> verticalSum(TreeNode root){
        Map<Integer,Integer> map=new TreeMap<>();
        dfs(root,0,map);
        return new ArrayList<>(map.values());
    }
    private void dfs(TreeNode n,int c,Map<Integer,Integer> m){
        if(n==null)return;
        m.put(c,m.getOrDefault(c,0)+n.val);
        dfs(n.left,c-1,m);
        dfs(n.right,c+1,m);
    }
}
```

Time Complexity: O(n log n) **Space Complexity:** O(n)

21. Maximum Width with Sparse Tree Indices

Problem: Find the maximum complete-tree position width even when many internal positions are missing.

Example Input: root=[1,3,2,5,3,null,9]

Example Output: 4

Approach: Use conceptual indices and normalize each level by its first index.

Algorithm: Left child gets 2*i and right child gets 2*i+1; width is last-first+1.

Java Answer:

```
class Solution {
    static class P { TreeNode n; long i; P(TreeNode n,long i){this.n=n;this.i=i;} }
    public int width(TreeNode root){
        Queue<P> q=new LinkedList<>();
        q.offer(new P(root,0));
        long best=0;
        while(!q.isEmpty()){
            int size=q.size();
            long base=q.peek().i, first=0,last=0;
            for(int j=0;j<size;j++){
                P p=q.poll();
                long idx=p.i-base;
                if(j==0)first=idx;
                if(j==size-1)last=idx;
                if(p.n.left!=null)q.offer(new P(p.n.left,2*idx));
                if(p.n.right!=null)q.offer(new P(p.n.right,2*idx+1));
            }
            best=Math.max(best,last-first+1);
        }
        return (int)best;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(w)

22. Expression Tree Evaluation

Problem: Evaluate a binary expression tree. Leaves contain integers and internal nodes contain operators +, -, *, or /.

Example Input: root represents ((2+3)*4)

Example Output: 20

Approach: Evaluate children first because operators depend on their operands.

Algorithm: If leaf return its number; otherwise evaluate left and right and apply the operator.

Java Answer:

```
class Solution {
    public int eval(TreeNode root){
        if(root.left==null&&root.right==null)return root.val;
        int l=eval(root.left), r=eval(root.right);
        // Assume operator is stored separately in an extended node class.
        return apply(root.op,l,r);
    }
    private int apply(char op,int a,int b){
        if(op=='+')return a+b;
        if(op=='-')return a-b;
        if(op=='*')return a*b;
        return a/b;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(h)

23. Tree Isomorphism with Swaps

Problem: Determine whether two binary trees have the same values and structure when children may be swapped independently at any node.

Example Input: A=[1,2,3], B=[1,3,2]

Example Output: true

Approach: At every pair of nodes, either children correspond directly or crosswise.

Algorithm: Compare current values and recursively test both direct and swapped child pairings.

Java Answer:

```
class Solution {
    public boolean isIsomorphic(TreeNode a, TreeNode b){
        if(a==null||b==null)return a==b;
        if(a.val!=b.val)return false;
        boolean direct=isIsomorphic(a.left,b.left)
            &&isIsomorphic(a.right,b.right);
        boolean swap=isIsomorphic(a.left,b.right)
            &&isIsomorphic(a.right,b.left);
        return direct||swap;
    }
}
```

Time Complexity: O(n) for unique/canonicalized variants; naive can branch exponentially **Space Complexity:** O(h)

24. Tree Diameter with Weighted Edges

Problem: Given a weighted tree, find the maximum total edge weight between any two nodes.

Example Input: edges=[[0,1,4],[1,2,3],[1,3,5]]

Example Output: 8

Approach: The diameter through a node is the sum of its two largest downward weighted paths.

Algorithm: Postorder returns the best downward weighted path and updates the global maximum.

Java Answer:

```
class Solution {
    List<int[]>[] g;
    long ans=0;

    public long diameter(int n,int[][] edges){
        g=new ArrayList[n];
        for(int i=0;i<n;i++)g[i]=new ArrayList<>();
        for(int[] e:edges){
            g[e[0]].add(new int[]{e[1],e[2]});
            g[e[1]].add(new int[]{e[0],e[2]});
        }
        dfs(0,-1);
        return ans;
    }
    private long dfs(int u,int p){
        long best1=0,best2=0;
        for(int[] e:g[u])if(e[0]!=p){
            long x=dfs(e[0],u)+e[1];
            if(x>best1){best2=best1;best1=x;}
            else if(x>best2)best2=x;
        }
        ans=Math.max(ans,best1+best2);
        return best1;
    }
}
```

Time Complexity: O(n) **Space Complexity:** O(n)